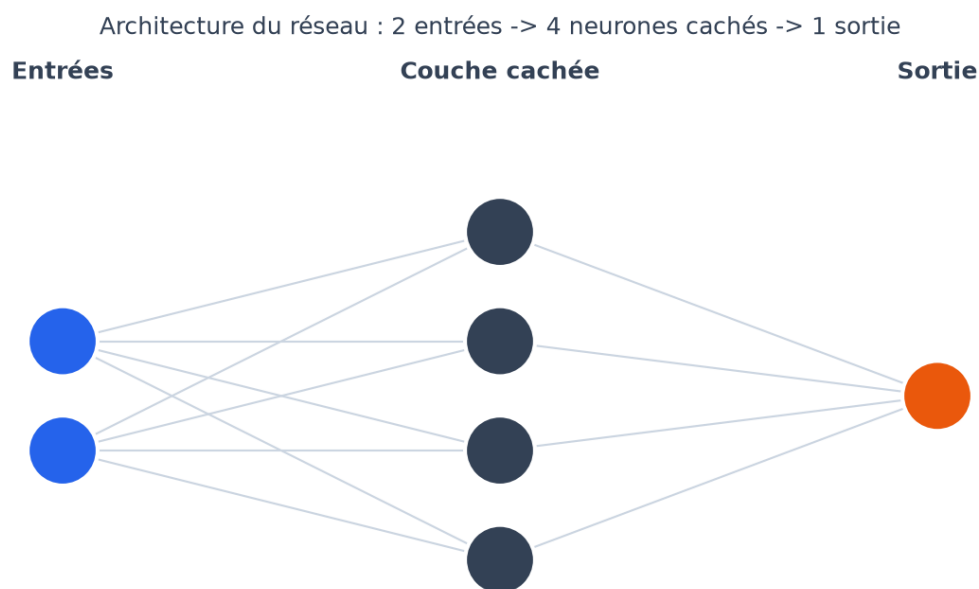


Comprendre la propagation dans un réseau de neurones

Forward propagation & backward propagation, expliquées pas à pas

Une implémentation from scratch en Python, avec NumPy uniquement



Ce document part de zéro : aucune connaissance préalable en apprentissage automatique n'est nécessaire. Chaque terme est défini au moment où il apparaît.

Sommaire

1. L'idée générale : qu'est-ce qu'un réseau de neurones ?
2. Un neurone, brique de base
3. La fonction d'activation
4. Les trois fonctions d'activation à connaître : sigmoïde, tanh, ReLU
5. La forward propagation (propagation avant)
6. Mesurer l'erreur : la fonction de coût
7. Apprendre = descendre la pente (descente de gradient)
8. La backward propagation (rétropropagation)
9. L'algorithme complet, en une page
10. Exemple concret : le problème du XOR
11. Le code Python commenté
12. Glossaire des termes importants

1. L'idée générale

Un **réseau de neurones** est une machine à faire des prédictions. On lui donne des **entrées** (par exemple les pixels d'une image, ou deux nombres 0 et 1) et il produit une **sortie** (par exemple « c'est un chat » ou « la réponse est 1 »).

Au départ, le réseau se trompe presque tout le temps, car il est rempli de nombres au hasard appelés **poids**. Tout l'enjeu de l'**entraînement** est de corriger progressivement ces poids pour que les prédictions deviennent justes. Ce processus repose sur deux mouvements qui se répètent des milliers de fois :

- **La forward propagation** : on pousse les entrées à travers le réseau pour obtenir une prédiction.
- **La backward propagation** : on compare la prédiction à la bonne réponse, puis on remonte le réseau à l'envers pour savoir comment corriger chaque poids.

En une phrase

Forward = **deviner**. Backward = **apprendre de son erreur**. On répète, et le réseau devient de plus en plus juste.

Imaginez un archer : il tire une flèche (forward), regarde de combien il a raté la cible (l'erreur), puis ajuste son geste pour le prochain tir (backward). Après beaucoup d'essais, il vise juste.

2. Un neurone, brique de base

Un **neurone** reçoit plusieurs nombres en entrée. Il multiplie chaque entrée par un **poids** (son importance), additionne le tout, puis ajoute un petit décalage appelé **biais**. On obtient un nombre intermédiaire noté z :

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Ici, les x sont les entrées, les w (weights) sont les poids, et b (bias) est le biais. C'est une simple **somme pondérée** : les poids disent à quel point chaque entrée compte, en bien (poids positif) ou en mal (poids négatif).

À retenir

Un neurone, c'est juste une multiplication suivie d'une addition. Rien de magique : ce sont les **bonnes valeurs** des poids (trouvées pendant l'entraînement) qui rendent le réseau intelligent.

3. La fonction d'activation

Si on empilait uniquement des sommes pondérées, tout le réseau resterait une grosse opération linéaire (une droite), incapable d'apprendre des formes compliquées. Pour lui donner de la souplesse, on fait passer z dans une **fonction d'activation**. La plus classique est la **sigmoïde** :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

La sigmoïde écrase n'importe quel nombre dans l'intervalle **$]0 ; 1[$** . Un grand nombre positif donne une valeur proche de 1, un grand nombre négatif une valeur proche de 0, et 0 donne exactement 0,5. C'est pratique : la sortie ressemble alors à une **probabilité**.

La valeur produite par le neurone après activation est notée a :

$$a = \sigma(z)$$

Pourquoi c'est indispensable

Sans activation, mille couches ne valent pas mieux qu'une seule. L'activation (non-linéaire) est ce qui permet au réseau de tracer des frontières courbes et de résoudre des problèmes que ligne droite ne peut pas séparer.

4. Les trois fonctions d'activation à connaître

On vient de voir la sigmoïde. En pratique, la fonction d'activation se choisit dans une petite « boîte à outils ». Voici les **trois noms** que vous croiserez tout le temps dans un cours ou un article : il faut savoir les reconnaître et comprendre à quoi chacune sert. Ce sont simplement trois façons différentes de « plier » le nombre z produit par un neurone.

La sigmoïde

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Se prononce « sig-moïde ». Elle écrase n'importe quel nombre **entre 0 et 1**. On l'utilise surtout sur le **dernier** neurone, quand on veut une sortie qui ressemble à une probabilité (par exemple « 87 % de chances que ce soit un chat »). Son défaut : dans les réseaux profonds, elle a tendance à **ralentir l'apprentissage**.

La tangente hyperbolique (tanh)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Se prononce « tan-ache », abréviation de « tangente hyperbolique ». C'est la cousine de la sigmoïde, mais elle écrase le nombre **entre -1 et 1** (elle est donc centrée sur 0). Dans les couches cachées, on la préfère souvent à la sigmoïde car le réseau apprend un peu plus vite.

ReLU (unité linéaire rectifiée)

$$\text{ReLU}(z) = \max(0, z)$$

Se prononce « ré-lou ». C'est l'abréviation de l'anglais *Rectified Linear Unit* (« unité linéaire rectifiée »). C'est la plus utilisée aujourd'hui dans les couches cachées, et pourtant la plus simple : elle **garde le nombre s'il est positif**, et le **remplace par 0 s'il est négatif**. Rapide à calculer et sans le défaut de lenteur de la sigmoïde, c'est le **choix par défaut** des réseaux modernes.

Les trois fonctions d'activation à connaître

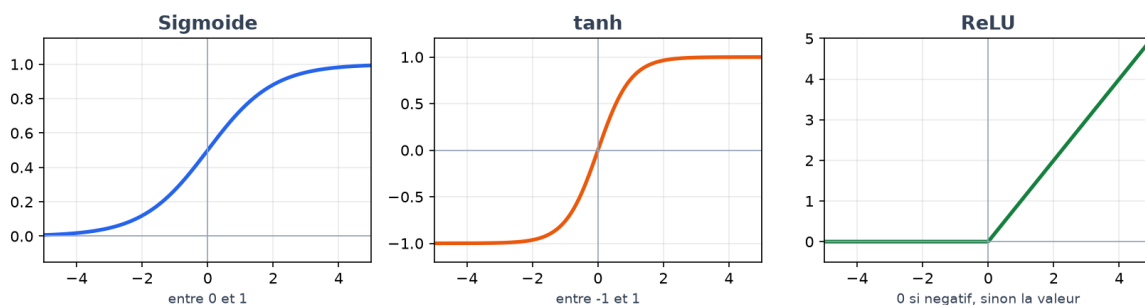


Figure — Les trois fonctions côte à côte. Notez la forme : la sigmoïde et tanh « écrasent », ReLU coupe simplement le négatif à zéro.

Comment choisir ? (le réflexe à retenir)

- **Couches cachées** : ReLU par défaut (simple et rapide).
- **Dernier neurone**, si on veut une probabilité : sigmoïde.
- **tanh** : une bonne alternative à la sigmoïde dans les couches cachées.

Dans notre exemple (section 10), on utilise la **sigmoïde partout**, uniquement pour n'avoir qu'une seule formule à expliquer. Le principe reste identique quelle que soit la fonction choisie.

5. La forward propagation (propagation avant)

On travaille rarement neurone par neurone : on regroupe les calculs de toute une couche dans une seule **multiplication de matrices**, ce qui est beaucoup plus rapide. Voici les quatre étapes pour notre réseau à une couche cachée (X contient les entrées, empilées en lignes).

Étape 1 — combinaison linéaire de la couche cachée

$$Z^{[1]} = X \cdot W^{[1]} + b^{[1]}$$

Étape 2 — activation de la couche cachée

$$A^{[1]} = \sigma(Z^{[1]})$$

Étape 3 — combinaison linéaire de la couche de sortie

$$Z^{[2]} = A^{[1]} \cdot W^{[2]} + b^{[2]}$$

Étape 4 — activation finale : la prédiction

$$A^{[2]} = \sigma(Z^{[2]}) = \hat{y}$$

Le symbole $\cdot$ représente le produit matriciel. Les exposants entre crochets, comme $W^{[1]}$, indiquent le numéro de la couche (1 = cachée, 2 = sortie). Le chapeau sur y signifie « valeur prédite », par opposition à y , la vraie réponse.

À retenir

La forward propagation, c'est simplement enchaîner : **multiplier par les poids** → **ajouter le biais** → **activer**, couche après couche, jusqu'à obtenir la prédiction.

6. Mesurer l'erreur : la fonction de coût

Une fois la prédiction obtenue, il faut chiffrer « à quel point on s'est trompé ». C'est le rôle de la **fonction de coût** (ou fonction de perte). Pour une sortie qui vaut 0 ou 1, on utilise l'**entropie croisée binaire** :

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Ici m est le nombre d'exemples. La formule punit fortement les prédictions confiantes mais fausses (par exemple prédire 0,99 alors que la vérité est 0). Quand la prédiction colle à la vérité, le coût tend vers 0.

Intuition

Le coût est une **note d'erreur** : plus il est bas, mieux le réseau prédit. Tout l'entraînement consiste à faire baisser ce nombre.

7. Apprendre = descendre la pente

Comment corriger les poids pour faire baisser le coût ? On utilise la **descente de gradient**. Imaginez le coût comme une vallée : chaque poids est une position, et on veut atteindre le fond (l'erreur minimale).

Le **gradient** d'un poids, noté $\partial L / \partial w$, indique la **pente** : dans quel sens et de combien le coût augmente si on augmente ce poids. Pour descendre, il suffit d'aller dans le sens **opposé** :

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

La lettre grecque η (êta) est le **taux d'apprentissage** : la taille du pas. Trop grand, on saute par-dessus le fond de la vallée ; trop petit, l'apprentissage est très lent. C'est un réglage à doser.

Image mentale

Vous êtes dans le brouillard sur une colline et voulez rejoindre le fond. Vous tâtez le sol pour sentir la pente (le gradient) et faites un pas vers le bas. Répété, ce geste vous mène au point le plus bas.

8. La backward propagation (rétropropagation)

Le problème : le coût dépend des poids de la sortie, qui dépendent des poids de la couche cachée, qui dépendent des entrées... Pour trouver l'influence de *chaque* poids sur le coût, on utilise la **règle de la chaîne** (dérivation en cascade) :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Autrement dit, on multiplie les petites influences le long du chemin. On part de l'erreur à la sortie et on la **propage vers l'arrière**, couche par couche — d'où le nom. On aura besoin de la dérivée de la sigmoïde, qui s'exprime très simplement à partir de sa sortie :

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)) = a (1 - a)$$

Les formules, de la sortie vers l'entrée

Couche de sortie. Avec une sortie sigmoïde et l'entropie croisée, la première dérivée se simplifie de façon spectaculaire :

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} A^{[1] \top} \cdot dZ^{[2]}$$

$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

Couche cachée. On renvoie l'erreur vers l'arrière en la multipliant par les poids de sortie, puis on traverse l'activation. Le petit rond barré dans la formule ci-dessous désigne une multiplication **terme à terme** (chaque case avec la case correspondante), à ne pas confondre avec le produit matriciel :

$$dZ^{[1]} = (dZ^{[2]} \cdot W^{[2] \top}) \odot \sigma'(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} X^{\top} \cdot dZ^{[1]}$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

Les quantités notées dW et db sont exactement les **gradients** attendus par la descente de gradient de la section 7. Une fois calculés, on met à jour tous les poids et biais, et on recommence.

À retenir

Backward = **remonter l'erreur à l'envers** pour attribuer à chaque poids sa part de responsabilité, grâce à la règle de la chaîne. Le résultat, ce sont les gradients qui disent comment corriger.

9. L'algorithme complet, en une page

Entraîner le réseau, c'est répéter la boucle suivante des milliers de fois. Une répétition complète s'appelle une **époque** :

- 1 **Forward** — Calculer la prédiction à partir des entrées (section 5).
- 2 **Coût** — Mesurer l'erreur entre prédiction et vérité (section 6).
- 3 **Backward** — Calculer les gradients de chaque poids par rétropropagation (section 8).
- 4 **Mise à jour** — Corriger chaque poids d'un petit pas dans le sens opposé à son gradient (section 7).
- 5 **Répéter** — Recommencer à l'étape 1 jusqu'à ce que le coût soit suffisamment bas.

Le cœur de tout l'apprentissage profond

Cette boucle « deviner → mesurer l'erreur → corriger » est **exactement** ce qui fait fonctionner les plus grands modèles d'intelligence artificielle. Ils ont plus de couches et de neurones, mais le principe est identique.

10. Exemple concret : le problème du XOR

Le **XOR** (« OU exclusif ») renvoie 1 quand les deux entrées sont **différentes**, et 0 quand elles sont identiques. C'est l'exemple historique : un neurone seul **ne peut pas** le résoudre, mais un réseau avec une couche cachée le résout parfaitement.

Entrée 1	Entrée 2	Sortie attendue
0	0	0
0	1	1
1	0	1
1	1	0

Après **10 000 époques** d'entraînement, le coût s'effondre : le réseau apprend. La courbe ci-dessous montre l'erreur qui diminue au fil du temps.

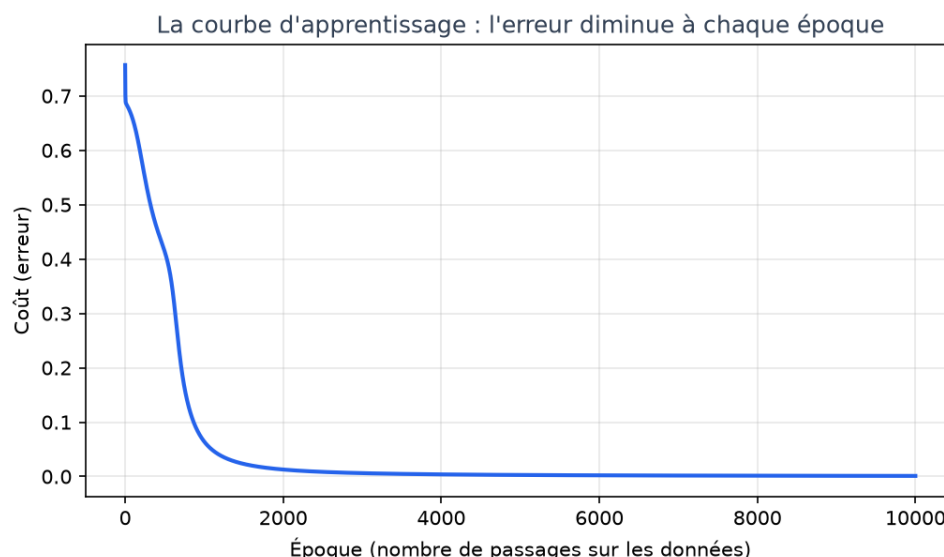


Figure — Le coût (l'erreur) chute rapidement puis se stabilise près de zéro.

On peut aussi visualiser ce que le réseau a « compris » en coloriant le plan selon sa prédiction. Les zones rouges correspondent à une sortie proche de 1, les zones bleues à une sortie proche de 0. On retrouve bien le motif en damier caractéristique du XOR.

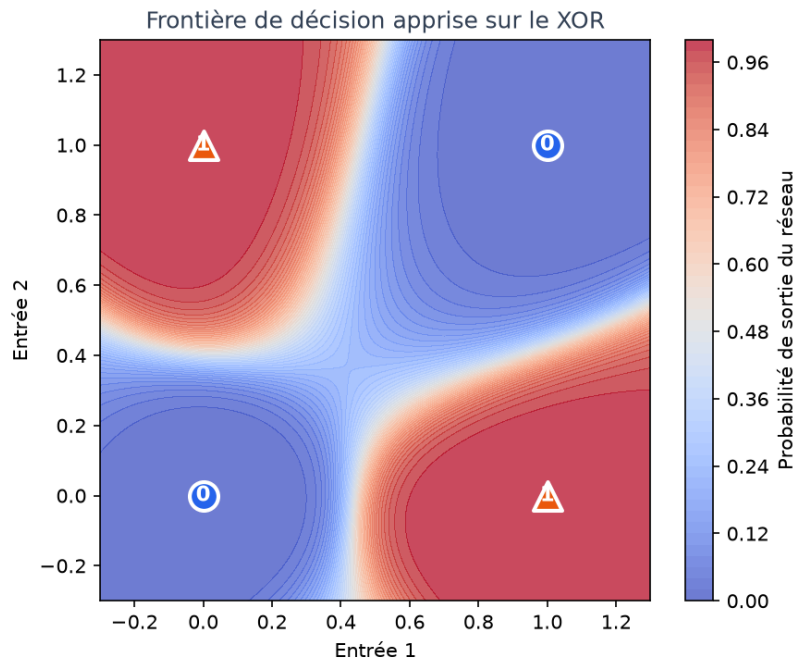


Figure — La frontière de décision. Les triangles oranges (sortie 1) et les cercles bleus (sortie 0) sont correctement séparés.

Résultat final obtenu par le programme :

Entrée	Probabilité prédite	Prédiction	Attendu
(0, 0)	0,001	0	0
(0, 1)	0,999	1	1
(1, 0)	0,999	1	1
(1, 1)	0,002	0	0

Exactitude : 100 %

Le réseau classe correctement les quatre cas. Parti de poids aléatoires, il a « appris » le XOR uniquement en répétant forward + backward.

11. Le code Python commenté

Voici le cœur de l'implémentation. Le fichier complet, abondamment commenté, accompagne ce document (`reseau_de_neurones.py`). Remarquez comme le code suit exactement les formules des sections précédentes.

La forward propagation en Python

```
def forward(self, X):
    # Couche cachee : combinaison lineaire puis activation
    self.Z1 = X @ self.W1 + self.b1
    self.A1 = sigmoid(self.Z1)
    # Couche de sortie : idem, la sortie est la prediction
    self.Z2 = self.A1 @ self.W2 + self.b2
    self.A2 = sigmoid(self.Z2)
    return self.A2
```

La backward propagation en Python

```
def backward(self, Y):
    m = Y.shape[0]
    # --- Couche de sortie ---
    dZ2 = self.A2 - Y
    dW2 = (self.A1.T @ dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m
    # --- Couche cachee (regle de la chaine) ---
    dA1 = dZ2 @ self.W2.T
    dZ1 = dA1 * sigmoid_derivee(self.A1)
    dW1 = (self.X.T @ dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m
    # --- Mise a jour des poids (descente de gradient) ---
    self.W2 -= self.taux_apprentissage * dW2
    self.b2 -= self.taux_apprentissage * db2
    self.W1 -= self.taux_apprentissage * dW1
    self.b1 -= self.taux_apprentissage * db1
```

Le symbole `@` est le produit matriciel de NumPy, et `.T` transpose une matrice (échange lignes et colonnes). Ces deux fonctions font tout le travail lourd à notre place.

12. Glossaire des termes importants

Poids (weight)	Nombre qui règle l'importance d'une entrée. C'est ce que le réseau apprend.
Biais (bias)	Décalage ajouté à la somme pondérée, qui permet de déplacer la fonction.
Fonction d'activation	Fonction non-linéaire qui donne au réseau sa souplesse. Les trois classiques : sigmoïde, tanh, ReLU.
Sigmoïde	Fonction d'activation qui écrase un nombre entre 0 et 1. Utile pour produire une probabilité en sortie.
tanh	« Tangente hyperbolique ». Comme la sigmoïde mais entre -1 et 1 ; souvent préférée dans les couches cachées.
ReLU	« Rectified Linear Unit ». Garde le nombre s'il est positif, sinon renvoie 0. Le choix par défaut des réseaux modernes.
Forward propagation	Passage des entrées vers la sortie pour produire une prédiction.
Fonction de coût	Mesure chiffrée de l'erreur entre prédiction et vérité.
Gradient	Pente du coût par rapport à un poids : dans quel sens et de combien le corriger.
Descente de gradient	Méthode qui corrige les poids en suivant la pente vers le bas.
Backward propagation	Calcul des gradients en remontant le réseau à l'envers (règle de la chaîne).
Taux d'apprentissage	Taille du pas de correction. À régler avec soin.
Époque	Une itération complète : forward + coût + backward + mise à jour sur toutes les données.

Félicitations

Vous connaissez maintenant les deux mécanismes qui font battre le cœur de tout réseau de neurones. Le reste de l'apprentissage profond n'est qu'une extension de ces mêmes idées.